

你验证的不只是代码 —— 还有 AgentOS 自身

业务验证

验证对象	错题本代码
问什么	「识别功能工作吗？」
谁出错改谁	代码 bug → 改代码
频率	每次 BUILD / VERIFY

系统验证 (Eval)

验证对象	AgentOS 行为
问什么	「agent 按我的标准工作吗？」
谁出错改谁	行为偏差 → 改 governance / engine
频率	每周 / 每次蒸馏后

本节内容 = 两种验证：代码对不对（业务） + 系统行为对不对（Eval）

AI 输出不确定 —— 怎么判定「对不对」

同一张图片两次调用可能返回不同文本，「正确」不是 exact match —— 是「足够好」。分层验证：

策略	适用场景	怎么做	类型
Schema 验证	输出格式	JSON schema 必须正确	确定性
关键字段验证	核心字段	confidence 存在、在 [0,1]	确定性
相似度验证	内容质量	识别文本与标注 $\geq 80\%$ 相似	阈值型

```
# ci/verify.sh - 业务验证 (课上只做确定性部分)
RESULT=$(python -c "from recognize import recognize; \
    import json; print(json.dumps(recognize('test.jpg')))" )
echo "$RESULT" | python -c "
import json, sys; d = json.load(sys.stdin)
assert 'confidence' in d, 'Missing confidence'
assert 0 <= d['confidence'] <= 1, 'Out of range'
assert 'question_text' in d, 'Missing question_text'
print('✅ Schema valid')"
```

核心理念

能确定性验证的部分，先做确定性验证。

别因「AI 输出不确定」就不做——你能验证的比想象的多。

Behavioral Contract = AgentOS 的「单元测试」

```
# eval/golden-set.md - Behavioral Contract
```

```
## 完成度行为
```

```
IF agent 声称完成一个功能  
THEN verify artifact 必须存在  
AND ≥ 5 种破坏尝试有记录  
AND 所有 AC 有 pass/fail 标记
```

```
## Engine 遵循
```

```
IF 任务类型是 feature  
THEN 经过 EVALUATE→PLAN→BUILD→VERIFY  
AND STATE.md 记录全部 gate 通过
```

```
## 蒸馏方向
```

```
IF corrections.log 有 3+ 条同类  
THEN governance/ 有对应 rule/principle
```

代码测试 vs 行为测试

代码测试

输入 X → 输出 Y 是否匹配 (测结果)

行为测试

过程 → 是否遵循了预期行为 (测过程)

Golden Set = 行为测试集

这些 IF-THEN 断言就是你的 golden set —— AgentOS 的行为测试集。

用它回答一个问题：我的系统在按我的标准运行吗？

把行为契约变成可执行的检查

```
#!/bin/bash # eval/run-eval.sh
PASS=0; FAIL=0
echo "=== AgentOS Behavioral Eval ==="

# 1. 完成度行为
for v in $(find spec -name 'verify*.md'); do
  A=$(grep -c '破坏尝试\|边界测试' "$v")
  [ "$A" -ge 5 ] && PASS=$((PASS+1)) \
    || FAIL=$((FAIL+1)); done

# 2. Engine 遵循 (STATE.md gate 记录)
# 3. Governance 覆盖 (principles ≥ 3)
# 4. 蒸馏健康 (active rules ≤ 15)

SCORE=$(( (PASS*100/(PASS+FAIL)) ))
echo " Score: ${SCORE}%"
```

1 · 完成度行为

verify artifact 是否有 ≥5 破坏尝试

2 · Engine 遵循

STATE.md 是否所有 gate 都过

3 · Governance 覆盖

principles 够不够、correction 有对应

4 · 蒸馏健康

rules 数量合理、有退休记录

输出一个分数。

100% 不是目标 —— 目标是「知道哪里不达标」。不达标 = 你的 AgentOS 待改进方向。

Agent 的行为会漂移 —— 怎么检测

Agent 今天遵循 principles, 下个月还遵循吗? 三个原因导致漂移:

漂移成因

- Context 变大 → principles 被稀释
- 新功能加 → rules 变多 → 互相冲突
- 没做蒸馏 → corrections 积累但没处理

漂移信号

- 同一类 correction 重复 (30 天内 ≥ 3 次)
- Gate 触发频率上升
- Eval score 下降
- agent 开始忽略某些 rules

漂移信号	响应
同类 correction 重复	蒸馏 → principle 不够锋利
Gate 触发上升	查 rules 被忽略 / context 膨胀
Eval score 下降	全面 health check → 找退化点
Rule 被忽略	升级为 Gate / 措辞改明确

建议频率: 每周跑一次 run-eval.sh, score 下降就做一轮蒸馏。

动手：写 Behavioral Contract + Eval 脚本

1

eval/golden-set.md

写 5-8 条 behavioral contract (IF-THEN)。覆盖：完成度、Engine 遵循、Principles 效果、蒸馏方向。

`eval/golden-set.md`

2

eval/run-eval.sh

至少实现 3 个自动化检查。可照抄基础版，也可扩展。

`eval/run-eval.sh`

3

ci/verify.sh

业务验证 (schema + 关键字段检查)。

`ci/verify.sh`

4

运行 eval

跑一次 run-eval.sh，看当前得分。分数不需要 100% —— 知道差距在哪。

→ Score: 72%

第一次 60-70% 很正常 —— 重要的是知道哪些行为还不达标，那就是接下来的改进方向。

本节总结

验证 = 代码对不对 + 系统行为对不对

✓ 业务验证

schema + 字段 + 相似度（分层策略）

✓ 系统验证（Eval）

Behavioral Contract + Golden Set + run-eval.sh + 漂移检测

关键理念：验证对象不只是代码——还有「系统按你的标准运行」这件事本身。

NEXT ▶

Loop Engineering

Engine 的自动化运行（茶歇后回来）